CSE344 - System Programming

Homework #1 Report

Muhammed Korkmaz
Student ID: 220104004043

March 12, 2026

1 Introduction

This report outlines the design and implementation of an "advanced" POSIX-compatible file search utility. The primary objective of this program is to recursively traverse a specified directory subtree and filter files based on user-defined criteria, which include filename pattern, size, type, permissions, and link count. The filtered results are then displayed in a visually structured, nicely formatted tree hierarchy.

2 Program Design

To achieve the exact tree-formatted output without printing empty or unmatched directories, an in-memory n-ary tree data structure (`TreeNode`) was designed.

- **Data Structure:** Every visited file and directory is stored as a node. A boolean flag (`has_match`) is propagated bottom-up from the leaves to the root. If a directory contains no matching files in any of its subtrees, it is completely pruned from the final output.
- **Argument Parsing:** Command-line arguments are securely handled using the `getopt()` library function.
- **Memory Management:** To strictly prevent memory leaks, a recursive `free_tree()` function was implemented. The program has been rigorously tested with Valgrind to ensure zero leaks under all conditions.
- **Signal Handling:** A `SIGINT` (CTRL-C) handler was integrated using `sigaction` to gracefully halt execution, free all dynamically allocated resources, and print an informative termination message before exiting.

3 Algorithm

The fundamental workflow of the program proceeds as follows:

1. Parse command-line parameters via `getopt()` and validate that the mandatory `-w` path is provided and accessible.

2. Initialize the root `TreeNode` for the target directory.

3. Recursively traverse the directory using `opendir()` and `readdir()`. Ignore `.` and `..` entries.

4. For each entry, retrieve its metadata using `lstat()` and evaluate it against the active search criteria (`-f`, `-b`, `-t`, `-p`, `-l`).

5. If an entry matches the criteria, mark its `is_match` flag as true, and trigger the `has_match` flag for all its parent nodes.

6. Recursively descend into subdirectories.

7. After traversal, recursively print the tree starting from the root, adjusting the indentation (number of dashes) based on the current depth. Nodes with `has_match == false` are ignored.

8. Free the entire tree structure and exit.

4 Pattern Matching Implementation

As the use of external regular expression libraries was prohibited, a custom `match_regex` function was implemented. The required `+` operator signifies one or more occurrences of the immediately preceding character.

The algorithm uses a two-pointer approach iterating over the pattern and the target string simultaneously. When a `+` is detected at the next index in the pattern, it consumes at least one matching character in the target string and continues to consume any consecutive identical characters. The comparison is strictly case-insensitive, accomplished by converting both characters to lowercase via `tolower()` before comparison.

5 System Calls and POSIX Functions Used

The program exclusively relies on standard POSIX functions for directory traversal and file I/O operations.

Function	**Purpose**
`opendir()`	Opens a directory stream for traversal.
`readdir()`	Reads successive entries from the directory stream.
`closedir()`	Closes the directory stream to prevent file descriptor leaks.
`lstat()`	Retrieves file metadata (size, type, permissions, links) without following symbolic links.
`getopt()`	Parses command-line arguments and flags.
`sigaction()`	Intercepts the `SIGINT` signal for graceful resource cleanup.

6 Error Handling

Robust error handling is implemented across all system calls. If a non-critical error occurs (e.g., lack of read permissions for a specific subdirectory), a nicely formatted

warning containing the `strerror(errno)` string is printed to `stderr`, and the traversal safely continues to other branches. For critical errors (e.g., memory allocation failure or an invalid root path), the program prints an informative message to `stderr` and immediately halts execution.

7 Conclusion

This implementation successfully fulfills all the requirements of an advanced POSIX file search utility. It accurately filters files based on various metadata combinations, handles regular expressions without third-party libraries, and securely manages memory and system resources. The hierarchical tree formatting provides a clean and readable output for the user.